



HỌC SÂU VÀ ỨNG DỤNG

Bài 2. Huấn luyện mạng Nơ-ron

Thông tin giảng viên:

TS. HUỲNH VĂN HUY

Điện thoại: **0914206920**

Email: **huynhvanhuy@gmail.com**



Bài 2. Huấn luyện mạng Nơ-ron

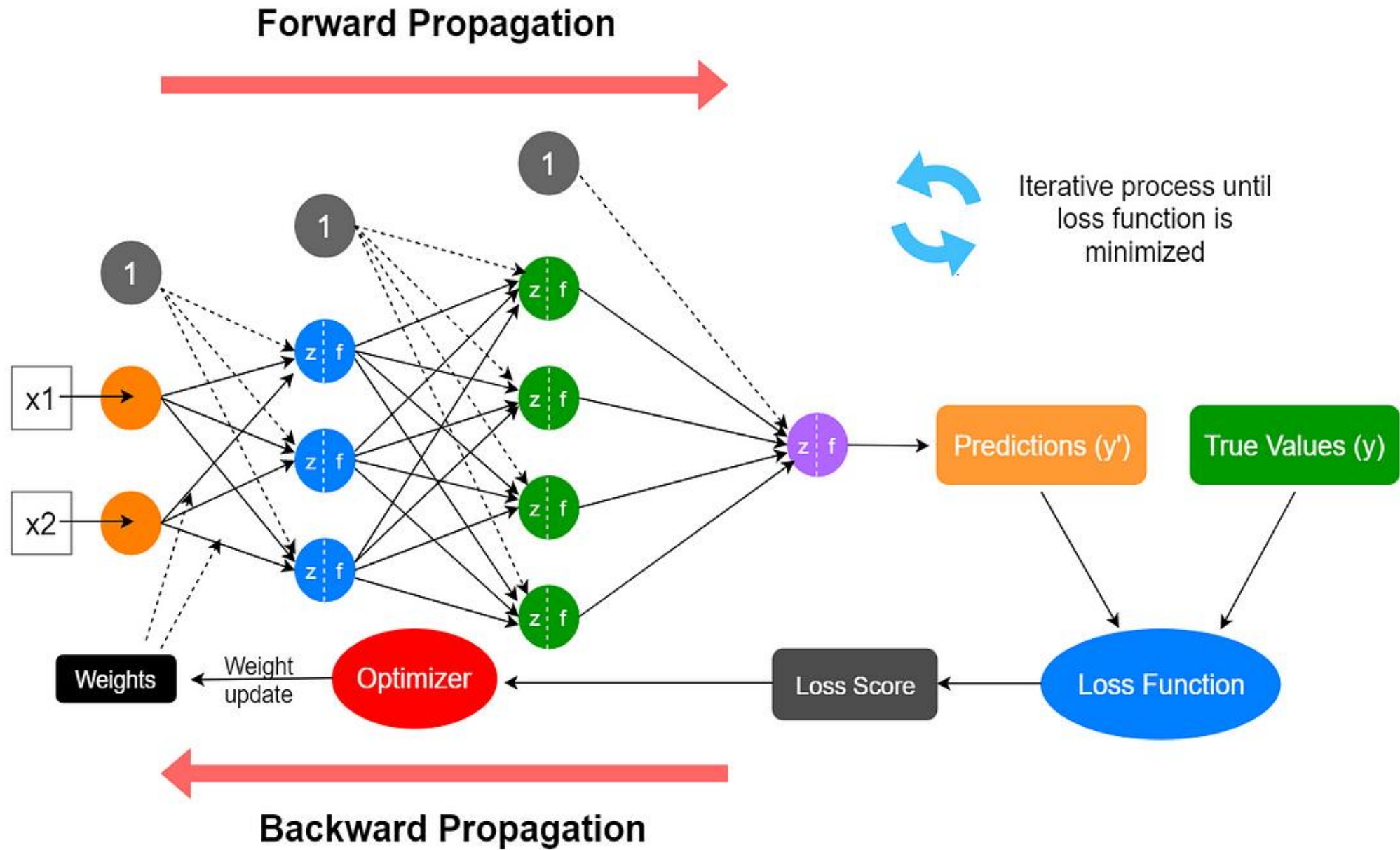
Mục tiêu bài học

1. Hiểu được nền tảng toán học về cơ chế lan truyền ngược trong học sâu
2. Hiểu được Các thuật toán tối ưu trong học sâu
3. Hiểu được Chiến thuật nạp dữ liệu & Siêu tham số
4. Thực hành: Xây mạng Nơ ron học sâu để Phân loại ảnh thời trang Fashion MNIST



1. Khái niệm Lan truyền ngược:

- Lan truyền ngược (Backpropagation) là thuật toán tính toán đạo hàm của hàm mất mát (Loss Function) theo từng trọng số (Weight) và độ lệch (Bias) trong mạng nơ-ron.
- Quá trình này diễn ra theo chiều từ lớp đầu ra (Output Layer) ngược về lớp đầu vào (Input Layer).





1. Khái niệm Lan truyền ngược:

- Mục tiêu của lan truyền ngược là xác định xem mỗi trọng số trong mạng đóng góp bao nhiêu phần trăm vào sai số tổng thể, từ đó cung cấp hướng đi để điều chỉnh trọng số.



2. Nền tảng toán học: Quy tắc dây chuyền (Chain Rule):

Giả sử một mạng nơ-ron đơn giản có 3 lớp, tạo thành hàm hợp: $y = f(g(h(x)))$

Để tính đạo hàm của hàm mất mát L theo trọng số w ở lớp đầu tiên, thuật toán áp dụng quy tắc dây chuyền của vi tích phân:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial g} \cdot \frac{\partial g}{\partial w}$$



3. Ví dụ minh họa tính toán

- **Bước 1 (Lan truyền tiến):** Đầu vào $x = 2$, trọng số $w = 3$. Hàm dự đoán $\hat{y} = w \cdot x = 6$.
Nhãn thực tế $y = 10$.

- **Bước 2 (Tính Loss):** Sử dụng hàm MSE: $L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(6 - 10)^2 = 8$.

- **Bước 3 (Lan truyền ngược):** Tính đạo hàm của L theo w :

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x = (6 - 10) \cdot 2 = -8$$



3. Ví dụ minh họa tính toán

- **Kết luận:** Đạo hàm mang giá trị âm (-8) , chỉ ra rằng để giảm sai số L , ta cần *tăng* giá trị của trọng số w .

Hãy tính lặp lại vòng 2, giả sử tăng trọng số w từ 3 ở bước khởi đầu lên thành 4, và tính lặp lại vòng 2 này, xem đạo hàm còn âm không?

Tính tới khi nào hàm mất mát tiến về cận 0 ($\text{Lim}0$) là Ok



4. Nghệ thuật điều hướng - Các thuật toán tối ưu (Optimizers)

Thuật toán tối ưu là cơ chế sử dụng đạo hàm (tính được từ Backpropagation) để cập nhật lại các trọng số, giúp mô hình trượt dần xuống điểm cực tiểu của hàm mất mát.



4. 1. SGD (Stochastic Gradient Descent - Hạ gradient ngẫu nhiên)

- **Nguyên lý:** Cập nhật trọng số bằng cách lấy giá trị hiện tại trừ đi đạo hàm nhân với tốc độ học (η).

$$w_{mới} = w_{cũ} - \eta \cdot \frac{\partial L}{\partial w}$$

- **Ưu điểm:** Đơn giản, dễ hiểu, tốn ít bộ nhớ.
- **Nhược điểm:** Dễ bị kẹt ở các điểm cực tiểu cục bộ (Local Minima) hoặc các điểm yên ngựa (Saddle Points). Quá trình di chuyển thường bị dao động mạnh (zigzag) nếu bề mặt hàm mất mát hẹp và dài.



Ví dụ: Ví dụ tính toán SGD theo phương pháp Hạ gradient ngẫu nhiên

1. Thiết lập bài toán

- Giả sử ta có một mô hình mạng nơ-ron cực kỳ đơn giản chỉ có 1 trọng số: $\hat{y} = w \cdot x$
- Hàm mất mát (Loss function) sử dụng MSE: $L = \frac{1}{2}(\hat{y} - y)^2$
- Đạo hàm của hàm mất mát theo trọng số: $\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x$
- Tốc độ học (Learning rate): $\eta = 0.1$
- Trọng số khởi tạo ban đầu: $w_{cũ} = 2$



Ví dụ: Ví dụ tính toán SGD theo phương pháp Hạ gradient ngẫu nhiên

Đặc trưng của SGD là chọn **ngẫu nhiên 1 mẫu dữ liệu** để tính toán cập nhật. Giả sử điểm dữ liệu được bốc ra có:

- Đầu vào: $x = 3$
- Nhãn thực tế mong đợi: $y = 12$



2. Các bước tính toán thực thi:

Bước 1: Lan truyền tiến (Forward Propagation)

Tính kết quả dự đoán của mô hình với trọng số hiện tại:

$$\hat{y} = w_{cũ} \cdot x = 2 \cdot 3 = 6$$

Bước 2: Lan truyền ngược (Backpropagation - Tính Gradient)

Tính mức độ lỗi và đạo hàm để tìm hướng cập nhật:

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x = (6 - 12) \cdot 3 = -18$$

(Ý nghĩa toán học: Gradient bằng -18 cho thấy bề mặt sai số đang dốc xuống. Thuật toán cần di chuyển ngược hướng âm (tức là đi theo hướng dương) để giảm sai số).



2. Các bước tính toán thực thi:

Bước 3: Cập nhật trọng số theo thuật toán SGD

Áp dụng công thức SGD cốt lõi:

$$w_{mới} = w_{cũ} - \eta \cdot \frac{\partial L}{\partial w}$$

$$w_{mới} = 2 - 0.1 \cdot (-18) = 2 + 1.8 = 3.8$$



3. Đánh giá kết quả "Học" :

Với trọng số mới $w_{mới} = 3.8$, nếu đưa lại mẫu dữ liệu $x = 3$ vào mô hình:

$$\hat{y}_{mới} = 3.8 \cdot 3 = 11.4$$

Dự đoán mới (11.4) đã tiến sát với nhãn thực tế (12) so với dự đoán ban đầu (6). Chỉ sau 1 vòng cập nhật duy nhất, thuật toán SGD đã giúp mạng nơ-ron "thông minh" hơn.



4. 2. SGD tích hợp Momentum (Động lượng)

- **Nguyên lý:** Khắc phục sự dao động của SGD bằng cách mô phỏng viên bi lăn xuống dốc. Nó cộng thêm một phần "đà" (velocity) của bước cập nhật trước đó vào bước cập nhật hiện tại.

$$v_t = \gamma v_{t-1} + \eta \cdot \frac{\partial L}{\partial w}$$

(Với γ thường được thiết lập ở mức 0.9).

- **Ưu điểm:** Giảm thiểu dao động, tăng tốc độ hội tụ và giúp thuật toán dễ dàng vượt qua các điểm cực tiểu cục bộ nông.



4. 3. **Adam** (Adaptive Moment Estimation)

Nguyên lý: Là thuật toán tiêu chuẩn và phổ biến nhất hiện nay. Adam kết hợp ưu điểm của Momentum (lưu lại trung bình của các đạo hàm trong quá khứ) và RMSprop (thích ứng tốc độ học cho từng trọng số riêng biệt dựa trên phương sai của đạo hàm).

Ưu điểm: Tự động điều chỉnh tốc độ học trong quá trình huấn luyện. Hội tụ cực kỳ nhanh và hoạt động hiệu quả với hầu hết các kiến trúc Học sâu.



Ví dụ về cách dùng thuật toán tối ưu Adam:

Với **Keras**:

Trong Keras, Adam được quản lý dưới dạng một đối tượng optimizer và được tích hợp vào mô hình thông qua phương thức **compile()**

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam
```

```
# 1. Khởi tạo mô hình tuần tự (Sequential Model)
model = Sequential(
    [
        Dense(50, activation='relu', input_shape=(10,)),
        Dense(1)
    ]
)
```

```
# 2. Định nghĩa thuật toán Adam với các siêu tham số tùy chỉnh
# Có thể sử dụng chuỗi 'adam' mặc định, hoặc khởi tạo object để tinh chỉnh
adam_optimizer = Adam(learning_rate=0.001, beta_1=0.9,
beta_2=0.999, epsilon=1e-7)
```

```
# 3. Biên dịch mô hình
model.compile(optimizer=adam_optimizer, loss='mse',
metrics=['mae'])
```

```
# 4. Huấn luyện mô hình với dữ liệu giả lập
import numpy as np
X_train = np.random.random((1000, 10))
y_train = np.random.random((1000, 1))
```

```
# model.fit(X_train, y_train, epochs=10, batch_size=32)
```



Ví dụ về cách dùng thuật toán tối ưu Adam:

Với **PyTorch**:

Trong PyTorch, thuật toán Adam được gọi thông qua module **torch.optim**. Quá trình này yêu cầu truyền các tham số của mô hình (model parameters) và tốc độ học (learning rate) vào bộ tối ưu.

```
import torch
import torch.nn as nn
import torch.optim as optim

# 1. Định nghĩa kiến trúc mạng neural
class SimpleNet(nn.Module):
    def __init__(self):
        super(SimpleNet, self).__init__()
        self.fc1 = nn.Linear(10, 50)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(50, 1)

    def forward(self, x):
        x = self.relu(self.fc1(x))
        return self.fc2(x)

model = SimpleNet()
```



Ví dụ về cách dùng thuật toán tối ưu Adam:

Với **PyTorch**:

Trong PyTorch, thuật toán Adam được gọi thông qua module **torch.optim**. Quá trình này yêu cầu truyền các tham số của mô hình (model parameters) và tốc độ học (learning rate) vào bộ tối ưu.

```
# 2. Khởi tạo thuật toán tối ưu Adam
# Các tham số mặc định phổ biến: lr=0.001, betas=(0.9,
0.999), eps=1e-08
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Định nghĩa hàm mất mát (Loss function)
criterion = nn.MSELoss()
```



Ví dụ về cách dùng thuật toán tối ưu Adam:

Với **PyTorch**:

Trong PyTorch, thuật toán Adam được gọi thông qua module **torch.optim**. Quá trình này yêu cầu truyền các tham số của mô hình (model parameters) và tốc độ học (learning rate) vào bộ tối ưu.

```
# 3. Cấu trúc một bước trong vòng lặp huấn luyện (Training step)
# Giả lập mini-batch dữ liệu đầu vào và nhãn
inputs = torch.randn(32, 10) # Batch size: 32, Features: 10
labels = torch.randn(32, 1)

# Xóa dải gradient tích lũy từ bước trước
optimizer.zero_grad()

# Lan truyền tiến (Forward propagation)
outputs = model(inputs)
loss = criterion(outputs, labels)

# Lan truyền ngược để tính gradient (Backward propagation)
loss.backward()

# Cập nhật trọng số dựa trên gradient vừa tính
optimizer.step()
```



Các siêu tham số:

Các siêu tham số (Hyperparameters) cốt lõi của Adam

- **Learning Rate (α hoặc `lr`):** Thông thường khởi tạo ở mức 0.001. Xác định độ lớn của bước cập nhật trọng số.
- **β_1 (Momentum của gradient bậc 1):** Mặc định 0.9. Kiểm soát tốc độ phân rã của trung bình động (exponential moving average) của các gradient trong quá khứ.
- **β_2 (RMSprop cho gradient bậc 2):** Mặc định 0.999. Kiểm soát tốc độ phân rã của trung bình động của bình phương các gradient.
- **ϵ (Epsilon):** Một hằng số rất nhỏ (thường là 10^{-8}) được cộng vào mẫu số để đảm bảo tính ổn định số học, tránh lỗi chia cho 0.



5. Chiến thuật nạp dữ liệu & Siêu tham số (Hyperparameters)

Siêu tham số là các thông số được thiết lập trước khi quá trình huấn luyện bắt đầu.



5. 1. Epochs (Số kỷ nguyên)

Định nghĩa: Một Epoch hoàn thành khi toàn bộ tập dữ liệu huấn luyện đã được đưa qua mạng nơ-ron đúng 1 lần (cả tiến và ngược).

Đặc điểm: Huấn luyện mạng nơ-ron đòi hỏi nhiều Epochs. Nếu số Epochs quá ít, mô hình sẽ bị Underfitting (chưa học đủ). Nếu số Epochs quá lớn, mô hình sẽ bị Overfitting (học vẹt dữ liệu).

Epoch đúng là một vòng lặp, nhưng nó là "**Vòng lặp lớn đi qua TOÀN BỘ tập dữ liệu**".



5. 1. Epochs (Số kỷ nguyên)

Trong quá trình huấn luyện AI, thực chất có tới 2 cấp độ vòng lặp lồng vào nhau. **Ví dụ về việc ôn thi:**

Giả sử bộ môn có một ngân hàng đề thi gồm **1000 câu hỏi** (Tương đương 1 Tập dữ liệu - Dataset).

1. **Vòng lặp nhỏ: Bước (Iteration / Step):**

Vì không thể nhồi nhét cùng lúc 1000 câu vào đầu, ta chia ngân hàng đề ra thành từng xấp nhỏ, mỗi xấp **100 câu** (Đây chính là **Batch Size**).



5. 1. Epochs (Số kỷ nguyên)

Lấy xấp đầu tiên (100 câu) ra học, làm bài, và đối chiếu đáp án để tự sửa lỗi sai của mình. Quá trình học xong 1 xấp này được gọi là **1 Bước (1 Iteration)**.

Trong code: tương ứng đây là lúc thuật toán tối ưu (như Adam, SGD) cập nhật trọng số 1 lần.



5. 1. Epochs (Số kỷ nguyên)

2. Vòng lặp lớn: Kỷ nguyên (Epoch)

Để học hết 1000 câu hỏi với mỗi lần học 100 câu, phải lặp lại quá trình trên **10 lần** ($1000 / 100 = 10$).

Khi đã nhìn thấy và làm qua **TOÀN BỘ 1000 câu hỏi đúng 1 lần duy nhất**, thì lúc đó ta gọi là hoàn thành **1 Epoch**.

Trong code: 1 Epoch chứa nhiều Iterations.



5. 1. Epochs (Số kỷ nguyên)

3. Ý nghĩa trong Học sâu:

Khi nói: "*Mô hình chạy 50 Epochs*", phải hiểu là: Cổ máy AI đã đọc đi đọc lại toàn bộ bộ sách giáo khoa (tập dữ liệu) **50 lần** từ trang đầu đến trang cuối.

Mỗi một lần đọc hết cuốn sách (1 Epoch), nó lại càng nhớ dai hơn và làm bài thi chính xác hơn. Nhưng nếu số Epochs quá lớn (đọc đi đọc lại 10.000 lần), nó sẽ chuyển sang học vẹt (Overfitting)



5. 2. Batch Size (Kích thước lô)

Định nghĩa: Thay vì tính toán đạo hàm cho toàn bộ tập dữ liệu (vốn không thể chứa vừa trong bộ nhớ GPU), tập dữ liệu được chia nhỏ thành các lô (Batch). Batch Size là số lượng mẫu dữ liệu có trong một lô.

Các chuẩn phổ biến: Thường là các lũy thừa của 2 (16, 32, 64, 128) để tối ưu hóa kiến trúc bộ nhớ phần cứng.



5. 2. Batch Size (Kích thước lô)

Tác động: Batch Size nhỏ giúp mô hình hội tụ nhanh hơn (do cập nhật trọng số liên tục) nhưng quỹ đạo loss sẽ bị nhiễu. Batch Size lớn cho đạo hàm chính xác hơn nhưng tốn nhiều bộ nhớ và thời gian cập nhật chậm.



5. 3. Learning Rate (Tốc độ học, ký hiệu là η)

Định nghĩa: Quyết định kích thước của "bước nhảy" khi cập nhật trọng số. Đây là siêu tham số quan trọng nhất.

Tác động:

Quá lớn: Bước nhảy vượt qua đáy thung lũng, hàm mất mát không bao giờ hội tụ, thậm chí phân kỳ (bắn lên vô cực).

Quá nhỏ: Mô hình mất quá nhiều thời gian để hội tụ, dễ bị kẹt vào cực tiểu cục bộ.

Lý tưởng: Bắt đầu bằng mức lớn (ví dụ: 0.001 với Adam) và giảm dần theo thời gian.



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.1. Đôi nét về ảnh thời trang Fashion MNIST:

Bộ dữ liệu **Fashion MNIST** được tạo ra và công bố vào năm **2017** bởi các nhà nghiên cứu đến từ **Zalando Research** (Zalando là một công ty thương mại điện tử đa quốc gia của Đức, chuyên về thời trang và giày dép). Nhóm tác giả chính bao gồm Han Xiao, Kashif Rasul và Roland Vollgraf.



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.1. Đôi nét về ảnh thời trang Fashion MNIST:

Lý do ra đời: Trước năm 2017, bộ dữ liệu MNIST gốc (chứa các chữ số viết tay từ 0-9) là tiêu chuẩn vàng để test các thuật toán AI. Tuy nhiên, giới nghiên cứu nhận ra rằng MNIST chữ số đã trở nên **quá dễ**. Các mạng nơ-ron đơn giản nhất cũng dễ dàng đạt độ chính xác trên 99%. Do đó, Zalando đã tạo ra Fashion MNIST như một "bản nâng cấp độ khó", thay thế trực tiếp chữ số bằng hình ảnh quần áo, nhưng vẫn giữ nguyên định dạng để các lập trình viên không phải sửa lại code cũ.



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.1. Đôi nét về ảnh thời trang Fashion MNIST:

Cấu trúc dữ liệu của bộ ảnh này: Fashion MNIST được thiết kế để trở thành một "bản sao y bản chính" của MNIST gốc về mặt định dạng (drop-in replacement), giúp người học cực kỳ dễ sử dụng:

Số lượng: Tổng cộng có **70.000** bức ảnh. Trong đó được chia sẵn thành:

Tập huấn luyện (Training set): 60.000 ảnh.

Tập kiểm tra (Testing set): 10.000 ảnh.



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.1. Đôi nét về ảnh thời trang Fashion MNIST:

Kích thước ảnh: Mỗi bức ảnh cực kỳ nhỏ, chỉ có kích thước **28x28 pixel** (tổng cộng 784 điểm ảnh).

Hệ màu: Tất cả đều là **ảnh xám (Grayscale)**. Mỗi điểm ảnh mang một giá trị số nguyên từ 0 (màu đen) đến 255 (màu trắng).



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.1. Đôi nét về ảnh thời trang Fashion MNIST:

Hệ thống phân loại (10 Nhãn/Classes)

Bộ dữ liệu chứa 10 loại trang phục/phụ kiện khác nhau, được đánh số (Label) chính xác từ 0 đến 9. Khi AI dự đoán, nó thực chất là đang cố gắng xuất ra một trong 10 con số này:

- 0: T-shirt/top (Áo thun/Áo phông)
- 1: Trouser (Quần dài)
- 2: Pullover (Áo len chui đầu)
- 3: Dress (Đầm/Váy liền)
- 4: Coat (Áo khoác)

- 5: Sandal (Dép Sandal)
- 6: Shirt (Áo sơ mi)
- 7: Sneaker (Giày thể thao)
- 8: Bag (Túi xách)
- 9: Ankle boot (Bốt cao đến cổ chân)



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.1. Đôi nét về ảnh thời trang Fashion MNIST:

Sử dụng bộ dữ liệu ảnh này có lợi ích:

Chạy được ngay trên CPU/Google Colab miễn phí:

Dữ liệu nhỏ gọn, tải về chỉ vài chục MB, huấn luyện mô hình tốn chưa tới 2 phút, không đòi hỏi phải có máy tính cấu hình khủng.

Khó hơn chữ số: Nhận dạng một cái áo thun (0) và một cái áo sơ mi (6) ở kích thước siêu mờ 28x28 pixel là một bài toán khó, buộc anh chị phải tối ưu hóa mô hình (như thêm lớp ẩn, đổi hàm kích hoạt, hay sau này là dùng mạng CNN) mới có thể đẩy độ chính xác lên cao.



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.2. Phát biểu bài toán:

Xây dựng mạng Nơ-ron đa lớp (MLP) để phân loại bộ dữ liệu Fashion MNIST. Yêu cầu: dùng Python, viết theo Keras, huấn luyện mô hình và trực quan hóa độ chính xác qua các Epochs, Vẽ đồ thị Loss và Accuracy và phân tích đôi nét về đồ thị để thấy rõ hiện tượng bắt đầu xuất hiện mô hình học sâu học vẹt?



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.3. Gợi ý các bước giải bài toán trên Python + Google Colab:

Bước 1: Khai báo thư viện và tải dữ liệu

Bước 2: Xây dựng kiến trúc Mạng nơ-ron

Bước 3: Thiết lập Thuật toán tối ưu và Hàm mất mát

Bước 4: Huấn luyện và thu thập lịch sử (History)

Bước 5: Vẽ đồ thị Loss và Accuracy



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.4. Thầy Huy gợi ý Coding căn bản dành cho các bạn yêu yêu Python:

Bước 1: Khai báo thư viện và tải dữ liệu



Bước 1: Khai báo thư viện và tải dữ liệu

```
import keras
```

```
import matplotlib.pyplot as plt
```

```
# Tải bộ dữ liệu Fashion MNIST từ Keras
```

```
fashion_mnist = keras.datasets.fashion_mnist
```

```
(train_images, train_labels), (test_images, test_labels) =  
fashion_mnist.load_data()
```

```
# Chuẩn hóa giá trị điểm ảnh về khoảng [0, 1] để tăng tốc hội tụ
```

```
train_images = train_images / 255.0
```

```
test_images = test_images / 255.0
```

```
# Khai báo danh sách tên các nhãn (0-9)
```

```
class_names = ['Áo thun', 'Quần dài', 'Áo len', 'Đầm', 'Áo khoác',  
               'Sandal', 'Áo sơ mi', 'Giày Sneaker', 'Túi xách', 'Ủng']
```



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.4. Thầy Huy gợi ý Coding căn bản dành cho các bạn yêu yêu Python:

Bước 2: Xây dựng kiến trúc Mạng nơ-ron



Bước 2: Xây dựng kiến trúc Mạng nơ-ron

```
model = keras.Sequential  
([  
    # Chuyển ảnh 2D (28x28) thành mảng 1D (784 điểm ảnh)  
    keras.layers.Flatten(input_shape=(28, 28)),  
  
    # Lớp ẩn: 128 nơ-ron, hàm kích hoạt ReLU  
    keras.layers.Dense(128, activation='relu'),  
  
    # Lớp đầu ra: 10 nơ-ron (tương ứng 10 loại quần áo), hàm  
    # kích hoạt Softmax  
    keras.layers.Dense(10, activation='softmax')  
)  
  
model.summary()
```



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.4. Thầy Huy gợi ý Coding căn bản dành cho các bạn yêu yêu Python:

Bước 3: Thiết lập Thuật toán tối ưu và Hàm mất mát



Bước 3: Thiết lập Thuật toán tối ưu và Hàm mất mát

```
# Sử dụng Optimizer Adam và hàm loss Sparse  
Categorical Crossentropy (dành cho nhãn chưa mã  
hóa One-hot)  
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.4. Thầy Huy gợi ý Coding căn bản dành cho các bạn yêu yêu Python:

Bước 4: Huấn luyện và thu thập lịch sử (History)



Bước 4: Huấn luyện và thu thập lịch sử (History)

```
# Huấn luyện trong 20 Epochs, sử dụng  
20% dữ liệu train để Validation (xác thực)  
print("Bắt đầu huấn luyện mô hình...")  
history = model.fit(train_images,  
train_labels,  
epochs=20,  
batch_size=32,  
validation_split=0.2,  
verbose=1)
```



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.4. Thầy Huy gợi ý Coding căn bản dành cho các bạn yêu yêu Python:

Bước 5: Vẽ đồ thị Loss và Accuracy



Bước 5: Vẽ đồ thị Loss và Accuracy

```
# Trích xuất dữ liệu từ object history
acc = history.history['accuracy']
val_acc = history.history['val_accuracy']
loss = history.history['loss']
val_loss = history.history['val_loss']
epochs_range = range(1, len(acc) + 1)

# Thiết lập biểu đồ
plt.figure(figsize=(14, 5))

# Đồ thị 1: Biểu diễn Accuracy
plt.subplot(1, 2, 1)
plt.plot(epochs_range, acc, label='Training Accuracy (Độ chính xác Train)')
plt.plot(epochs_range, val_acc, label='Validation Accuracy (Độ chính xác Val)')
plt.legend(loc='lower right')
plt.title('Độ chính xác qua các Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
```

```
# Đồ thị 2: Biểu diễn Loss
plt.subplot(1, 2, 2)
plt.plot(epochs_range, loss, label='Training Loss (Sai số Train)')
plt.plot(epochs_range, val_loss, label='Validation Loss (Sai số Val)')
plt.legend(loc='upper right')
plt.title('Hàm mất mát qua các Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')

plt.tight_layout()
plt.show()
```



6. Thực hành: Phân loại ảnh thời trang Fashion MNIST

6.5. Nhận xét sơ nét về đồ thị:

Phân tích đồ thị: Tại các biểu đồ này, đường biểu diễn Training luôn có xu hướng cải thiện (Loss giảm, Accuracy tăng). Tuy nhiên, đường biểu diễn Validation sẽ chững lại sau một số Epochs nhất định. Điểm chững lại đó chính là ranh giới bắt đầu xảy ra hiện tượng Overfitting.